

Assignment -1

Name : Imtiaz Hossain

ID : 23101137

Section: 04

Course : CSE 224
code

Faculty : [PNP]
Initial

Answer to the question no 1

(a)

for ($i=0 ; i < n ; i+=5$) { $\frac{n}{5} = n \rightarrow O(n)$
 for ($j=1 ; j < n ; j*=3$) { $\log_3 n \rightarrow O(n \log_3 n)$
 for ($k=0 ; k \leq 30 ; k++$) { $30 \rightarrow O(1)$
 $arr[i] = arr[i] + 1 ;$
 $O(n \log_3 n)$

}

int = 0
 for ($m=n ; m > 1 ; m/=2$) { $\log_2 n$
 $a = b + c ;$
 $\rightarrow O(n \log_3 n \times \log_2 n)$

}

In Big-O notation, $O(\log_3(n))$ can be simplified to $O(\log n)$ because logarithms of different bases differ only by a constant factor.

$$\therefore O(n \log_3 n) = O(n \log n)$$

$$\therefore O(n \log_3 n \times \log_2 n) = O(n \log n \times \log n)$$

$$\text{Here, } O(n \log n \times \log n) > O(n \log n)$$

So, the overall time complexity is:

$$O(n (\log n)^2)$$

(Ans)

(b)

for i in range (1, n): $\rightarrow (n-1) \rightarrow O(n)$

j = 1

while j < i * i: $\rightarrow (1 - \infty) \rightarrow O(\infty)$

j = j + 1

i	j
1	0
	-1
	-2
	-3
	...
	$-\infty$

This indicates that the code will run forever and never terminate, which theoretically indicates non-termination.

So, the overall time complexity is:

$O(\infty)$ (Ans)

(c)

k
16
1

j
1 (1 ² < 16)
3 (3² < 16)
5 (5² < 16)
...

i
1
4
16
64 x
...

z
1
5
21 < 16
...

m
1 (1 ² < 16)
4 (4² < 16)
...

let, n = 16

1 x

16
64 x
~~...~~

(c)

$$\text{Sort}(k=n; k>1; k=k/2) \left\{ \text{--- } \log_2 n = 1 \rightarrow O(1) \right.$$

$$\text{Sort}(j=1; j*j \leq n; j=j+2) \left\{ \text{--- } \sqrt{n} \rightarrow O(\sqrt{n}) \right.$$

$$\text{Sort}(i=1; i \leq n; i=i*4) \left\{ \text{--- } \log_4 n \rightarrow O(\sqrt{n} \log_4 n) \right.$$

$$z = z + i$$

}

}

$$\text{Sort}(m=1; m*m < n; m=m*4) \left\{ \text{--- } \log_4 n = \frac{1}{2} \log_2 n \right.$$

$$b = b + 1$$

$$\rightarrow O(\log_4 n)$$

}

}

In Big-O notation, $O(\log_4(n))$ can be simplified to $O(\log n)$ because logarithms of different bases differ only by a constant factor.

$$\therefore O(\log_4 n) = O(\log n)$$

$$\text{and, } O(\sqrt{n} \times \log_4 n) = O(\sqrt{n} \log n)$$

Here,

$$O(1) \times (O(\sqrt{n}) \times O(\log n) + O(\log n))$$

$$= O(\sqrt{n} \log n) + O(\log(n))$$

$$\therefore O(\sqrt{n} \log n) > O(\log n)$$

So, the overall time complexity is: $O(\sqrt{n} \log n)$

Ans

Answer to the question no: 2

(a)

$$T(n) = 8T(n/4) + n\sqrt{n}$$

If we use master Theorem, we can get running time $T(n)$ with an asymptotic bound.

We know that,

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

here,

$$a = 8$$

$$b = 4$$

$$f(n) = n\sqrt{n} = n^{3/2}$$

comparator : $n^{\log_b a}$

$$= n^{\log_4 8}$$

$$= n^{3/2}$$

$$\therefore f(n) = n^{\log_b a} = n^{3/2}$$

$$\therefore f(n) = \Theta(n^{\log_b a})$$

$$\therefore T(n) = \Theta(n^{\log_b a} \times \log n) = \Theta(n^{3/2} \log n)$$

so, asymptotic bound for $T(n)$ is:

$$T(n) = \Theta(n^{3/2} \log n)$$

(Ans)

(b)

$$T(n) = T(n-3) + n$$

If we use Iterative method, we can get running time $T(n)$ with an asymptotic bound.

$$T(n) = T(n-3) + n$$

$$= T(n-6) + (n-3) + n$$

$$= T(n-9) + (n-6) + (n-3) + n$$

⋮

continued for k times

$$= T(n-3k) + \cancel{(n-3(k-1))} + n + (n-3) + (n-6) + \dots + (n-3(k-1))$$

or,

$$= T(n-3k) + (n-3(k-1)) + (n-3(k-2)) + \dots + (n-6) + (n-3) + n$$

Let,

$$n-3k=0$$

$$\therefore k = \frac{n}{3}$$

$$\therefore T(n) = T\left(n-3 \times \frac{n}{3}\right) + (n-3\left(\frac{n}{3}-1\right)) + \dots + (n-6) + (n-3) + n$$

$$= T(n-n) + (n-n+3) + (n-n+6) + \dots + (n-6) + (n-3) + n$$

$$= T(0) + 3 + 6 + \dots + (n-6) + (n-3) + n$$

$$= 1 + 3 + 6 + \dots + (n-6) + (n-3) + n$$

$$= \frac{1(3^{\log_3 n} - 1)}{3 - 1}$$

$$= \frac{3^{\log_3 n} - 1}{2}$$

After 1,

this sequence is an arithmetic series, where

$$a = 3$$

$$d = 3$$

$\therefore k$ terms will be -

$$a_k = a + (k-1)d$$

$$\therefore a_k = 3 + (k-1)3$$

$$= 3 + 3k - 3 = 3k$$

$$3k = n$$

$$\therefore k = \frac{n}{3}$$

Sum of Arithmetic sequence:

$$S_k = \frac{k}{2} (a + l)$$

$$= \frac{\frac{n}{3}}{2} (3 + n)$$

$$= \frac{n}{6} (3 + n)$$

$$= \frac{n(n+3)}{6} = \frac{n^2 + 3n}{6} = \frac{n^2}{6} + \frac{n}{2}$$

Here,

$$a = 3$$

$$d = 3$$

$$k = \frac{n}{3}$$

$$\therefore T(n) = 1 + \left(\frac{n^2}{6} + \frac{n}{2} \right)$$

For large n , the dominant term is

$$\frac{n^2}{6}.$$

So, asymptotic bound for $T(n)$ is:

$$T(n) = \Theta(n^2)$$

(Ans)

(c)

$$T(n) = 625T(n/5) + n^3$$

If we use Master Theorem, we can get running time $T(n)$ with an asymptotic bound.

we know that,

$$T(n) = aT(n/b) + f(n)$$

here,

$$a = 625$$

$$b = 5$$

$$f(n) = n^3$$

comparator:

$$\begin{aligned} & n^{\log_b a} \\ &= n^{\log_5 625} \\ &= n^{(\log_5 5)^4} = n^{(1)^4} \\ &= 4n \qquad \qquad \qquad = n^4 \end{aligned}$$

$$\therefore f(n) < n^{\log_b a}$$

Since, $f(n)$ is polynomially smaller than $n^{\log_b a}$, by the Master Theorem,

$$\begin{aligned} T(n) &= \Theta(n^{\log_b a}) \\ &= \Theta(n^4) \end{aligned}$$

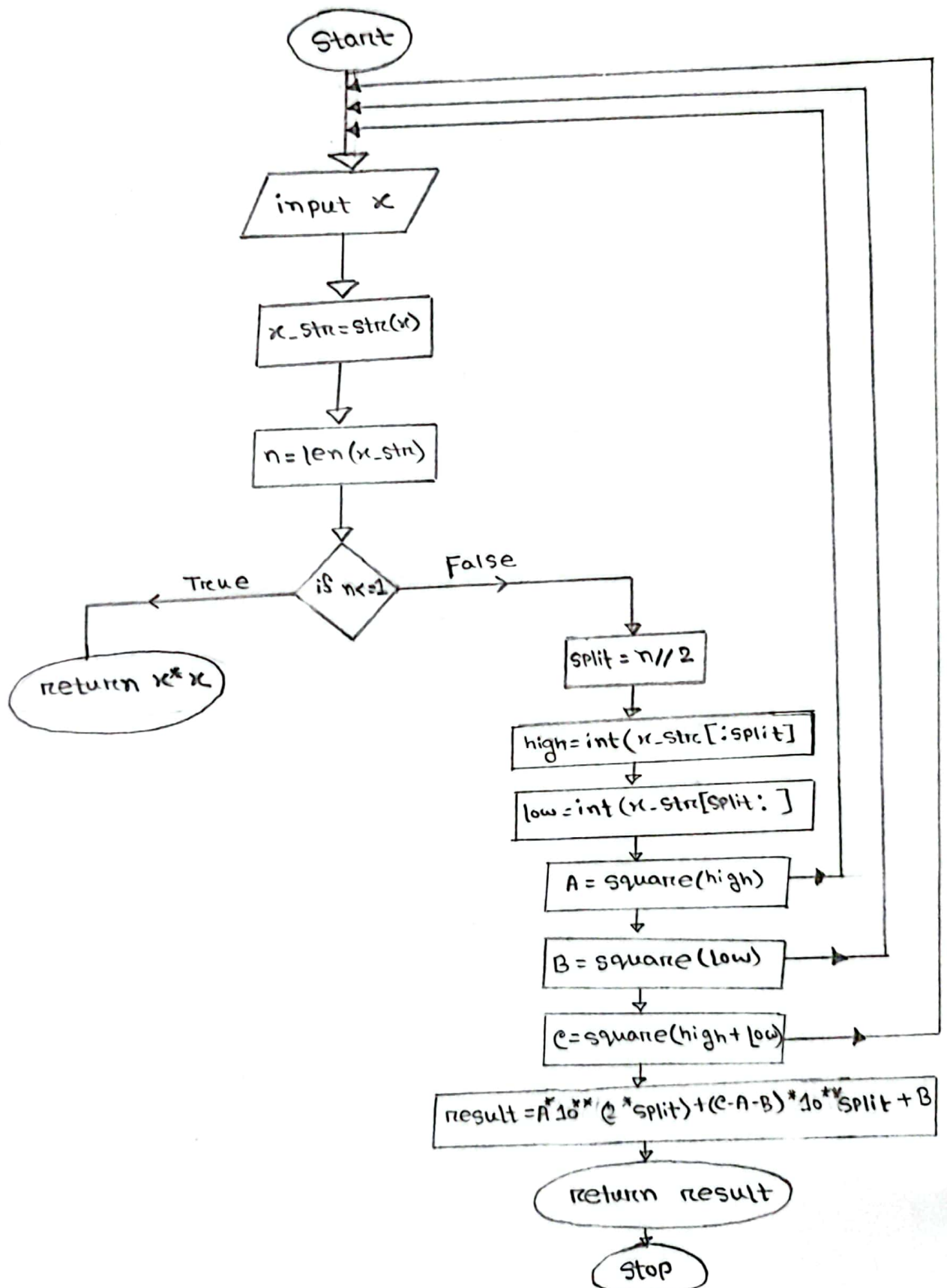
So, asymptotic bound for T is:

$$T(n) = \Theta(n^4)$$

(Ans)

Answer to the question no 3

(a)



Answer to the question no 3

(a)

Flowchart -

Start

- 1) Created a function wh. named `square()` which will take a parameter x .
- 2) Number x (with n digits) will be the input.
- 3) Base case: If $n \leq 1$, then it will return x^2 .
- 4) Divide : Firstly, it will split x into two parts:
 - Let, x_1 = high part and x_0 = low part by choosing a splitting point
- 5) Recursive calculation: we compute the square of $(x_1)^2$, $(x_0)^2$ and $(x_1 + x_0)^2$.

$$\begin{aligned} \text{Let, } A &= x_1^2 \\ B &= x_0^2 \\ C &= (x_1 + x_0)^2 \end{aligned}$$

- 6) Result : ^{then} we compute the result as -

$$A \times 10^{2 \times \text{[split]}} + (C - A - B) \times 10^{\text{[split]}} + B$$

- 7) Output: Output will be generated in the result and we will return the result.

End

(b)

```
def square(x):
```

```
    x_str = str(x)
```

```
    n = len(x_str)
```

```
    if n <= 1:
```

```
        return x * x
```

```
    split = n // 2
```

```
    high = int(x_str[:split])
```

```
    low = int(x_str[split:])
```

```
    A = square(high)
```

```
    B = square(low)
```

```
    C = square(high + low)
```

```
    result = A * 10**(2*split) + (C - A - B) * 10**split  
            + B
```

```
    return result
```

(c)

Time Complexity:

$$\text{Split} = n//2$$

- Divide step: Splitting the number into two halves takes constant time.

- Recursive steps:

A = square (high)

B = square (low)

- we compute the square of x_1 and x_0 which involves two recursive calls.

C = square (high + low)

- we also compute $(x_1 + x_0)^2$ which involves another recursive call.

Here, the number of recursive calls is similar to the Karatsuba algorithm's approach.

Each recursive call handles a problem of size $\frac{n}{2}$, and we make three recursive calls.

Thus, the recurrence relation for this algorithm is:

$$T(n) = 3T\left(\frac{n}{2}\right) + O(n)$$

Let's use Master Theorem here.

$$\therefore a = 3$$

$$b = 2$$

$$f(n) = \cancel{O(n)} = O(n)$$

$$\text{comparator: } n^{\log_b a}$$

$$= n^{\log_2 3}$$

$$= n^{1.585}$$

$$\therefore f(n) < n^{\log_b a}$$

Since, $f(n)$ is polynomially smaller than $n^{\log_b a}$, by the Master Theorem,

$$T(n) = O(n^{\log_2 3})$$

$$\approx O(n^{1.585})$$

(Ans)

space complexity : $O(n)$

— x —